

SAMVAAD

Learn by explaining. Be measured by understanding.

Cline AI Builders Hackathon | Round 1 | Theme: Reinventing Education with AI Target:
Grades 6-12, India, multilingual, large classes | **Team:** 1 builder (React/Python, Cline-driven)

This submission has two parts, as required: Part A (Ideation) and Part B (Technical Documentation). No prototype or code is included for this round.

PART A: IDEATION

A1. The idea

Samvaad (Hindi: *dialogue*) is a learning platform where **students teach a curious AI classmate**, and the quality of their explanation becomes the measure of their understanding.

Every student teaches an AI classmate. Every explanation becomes evidence of understanding. Every teacher sees who needs them today.

A2. The problem

Today's classroom measures whether a student can *recall a procedure*, not whether they *understand why it works*. Four structural failures follow:

1. **Tests reward recognition, not explanation.** MCQs and answer-matching cannot show whether a student has the underlying idea.
2. **Feedback is slow and coarse.** A mark says "wrong", not *why*. Notebook corrections return in days, or never.
3. **Pace follows the calendar, not the learner.** A missed idea in Class 7 becomes a wall in Class 9.
4. **Teachers cannot hear students think.** With 50+ students per section, a teacher finds out who was lost only after the test.

Example. Priya (Class 8, a section of 52) learns the rule "move the term across and change the sign" and scores 8/10. Two months later she gets $(3x = x + 8)$ wrong, because she never learned that an equation is a *balance*. The test never exposed this, her teacher saw only a mark, and Priya concluded "I am bad at algebra."

Why this matters: Research supports the direction of the fix: explaining improves understanding (self-explanation, the "protégé effect"), and spaced retrieval and formative feedback improve retention. It also warns that AI which does the thinking *for* students can reduce later performance (Bastani et al., PNAS 2025). Samvaad is designed around that warning.

A3. How it works

The unit of learning is **the explanation**. Students explain, an AI classmate who knows only what they said tries to use it, the result becomes evidence, and teachers see patterns.

For the student (about 15 minutes, on a shared phone, text or voice):

1. **Teach Mitra.** Mitra is a friendly, slightly confused classmate who holds a *specific misconception* (for example, "change the side, change the sign" with no idea why). The student explains the concept in Hindi, English or a mix.
2. **Test my explanation.** A second Mitra receives *only the student's words* plus a fresh problem. If the explanation is incomplete, Mitra gets stuck at a specific step ("you never said to do the same thing to both sides"). The student repairs the explanation and Mitra tries again.
3. **Notebook check.** The student photographs homework; the system points to the first wrong step, and the student confirms the reading.
4. **Return visits.** Days later Mitra says "I forgot, can you remind me?" This is a retention check framed as helping a friend, not an exam.

For the teacher (about 3 minutes before school): a short *Today's 5* list of students who need a human, each with a one-line reason and quoted evidence, plus one class-level insight such as "*17 of 52 apply 'change the sign' without the balance idea; suggested 8-minute activity attached.*" Teachers approve all content and can override any result.

A4. How AI is used

Capability	What it does	Why AI is essential
Mitra persona	Plays a classmate with a scripted misconception; asks natural follow-ups; never reveals the answer	Needs open-ended language generation
Explanation testing (core idea)	A model given <i>only the student's explanation</i> attempts a new problem; a symbolic verifier checks the answer; the failing step shows the gap in the explanation	Needs a reader of free-form explanations; no pre-AI equivalent
Evidence extraction	Reads an explanation and marks each required idea present, partial or absent, quoting the student's own words as proof	Free-text understanding at scale
Adaptive probing	Picks the next question to test the weakest idea; goes back to prerequisites if stuck	Responds to what the student actually said
Notebook and speech reading	Vision and speech models read handwritten steps and spoken Hindi/English	Impossible without multimodal AI
Teacher insight	Clusters misconceptions across a class and writes a short, evidence-cited summary	Understanding what students <i>said</i> , not just counting marks
Mastery and review scheduling	Classical statistics and a spaced-repetition scheduler, deliberately <i>not</i> an LLM	Kept inspectable so a child's status is never decided by a black box

A5. Rethinking measurement

Samvaad replaces one-shot marks with **continuous, low-stakes evidence** across five dimensions: *understanding* (required ideas present in explanations), *reasoning process* (where errors begin), *retention* (checks at about day 3, 7 and 21), *transfer* (fresh problems in new contexts) and *independence* (how much help was needed).

Each concept shows one of four statuses: **Not yet seen, Emerging, Secure, Needs refresh**, with a confidence level and a clickable evidence trail. "Secure" requires evidence from sessions about a week apart and an unaided transfer success. "Not enough evidence yet" is a valid answer.

What makes it trustworthy:

- **Quote-grounded.** Every claim of understanding must cite the student's own words, and code rejects any quote that is not actually in the text.

- **Scored by ideas, not fluency.** A broken-English explanation containing the idea scores the same as a polished one.
- **Calibrated against teachers.** A sample of explanations is double-scored by teachers every week; weak-agreement areas are marked "teacher-confirmed only."
- **Hard to game.** Follow-up questions are generated live from the student's own claims; transfer problems are fresh; voice, text and notebook evidence are cross-checked, and a mismatch triggers "talk to this student," never a penalty.
- **Contestable.** Students can tap "I disagree"; teachers can override any status.

Samvaad does not abolish board exams; it makes preparation diagnostic instead of last-minute.

A6. Role of the teacher

The teacher becomes **coach** (picks the lesson the class needs), **intervention specialist** (spends one-to-one time where evidence says it matters) and **mentor**. Teachers keep control of approving the concept graph and rubrics, overriding any status, and deciding what is said to parents.

A7. Responsible AI

Risk	Control
Hallucination	Curriculum facts come from approved content, not model memory; answer keys are verified by code; evidence needs quoted spans; Mitra never states the solution
Bias and fairness	Accuracy tested by language, dialect, gender and school type; scoring by ideas, not style; students are never ranked against each other
Student privacy	Minimum data, pseudonymous IDs, no ads or trackers, raw audio deleted after confirmation, parental consent via the school (aligned with India's DPDP Act, to be legally reviewed)
Over-reliance	The student does the explaining and the AI knows less by design; no full solutions on demand; pilot includes an AI-free post-test
Access	Light installable web app, voice and text equal, offline evidence queue, shared-device mode
Wellbeing	Concerning language is routed to a human counsellor, never handled by the AI alone

Human-in-the-loop points: teachers approve all concept graphs, rubrics and prompts; teachers calibrate weekly and can override any status; no parent "concern" messages without teacher review; every model, prompt or rubric change passes an evaluation suite and human sign-off.

A8. What is new

Most AI tutors make the AI the expert and the student the questioner. Samvaad **inverts the roles**: the student teaches an AI that does not know. Evidence of understanding comes from whether the student's own explanation *works* for a learner with nothing else to rely on. Combined with quote-grounded scoring and a "who needs me today" teacher view, this is a measurement system, not a tutor with extra steps.

A9. Impact and success metrics

Area	Metric
Learning gain	Concept test with explanation items, immediately and at +4 weeks
Independence	Performance on an AI-free paper post-test
Teacher benefit	Minutes per week on grading and planning; students receiving 1:1 attention
Trust	Teacher-AI agreement; override rate
Equity	Gain gap by language, gender and device access

A small pilot (a few class sections across 2-3 schools, randomised by section, 6-8 weeks) would test feasibility and signal. It is framed honestly as underpowered for small effects.

PART B: TECHNICAL DOCUMENTATION

B1. Architecture

Layers (request flow reads top to bottom):

Layer	Components
Clients	Student web app (installable, works offline for evidence), Teacher Cockpit, parent/admin views
API	FastAPI service with authentication, role checks and a consent check on every student-data request; streaming for Mitra's replies
Orchestration	A session state machine (explain, probe, execute, repair, wrap) that decides what happens next; Mitra engine; peer-execution engine; probe selector; evidence extractor; problem generator; notebook reader
AI services	One LLM gateway (routing, retries, caching, budgets, logging) in front of all models; speech-to-text, text-to-speech, vision model, embedding model; symbolic verifier (SymPy, sandboxed); moderation and PII scrubbing
Data	PostgreSQL with pgvector (system of record and curriculum embeddings); Redis (job queue, cache); object storage (notebook images, temporary audio)
Analytics and quality	Background workers for evidence extraction, mastery updates and clustering; evaluation harness that reuses the production gateway

Key design decisions:

- **LLMs fill content, never control flow.** The state machine is plain, testable logic.
- **LLMs produce evidence but never set mastery.** A classical, inspectable model does.
- **All model calls pass through one gateway**, so provider changes, cost limits and logging live in one place.
- **Slow work is asynchronous**, so students never wait on analytics.

B2. Core components

Component	Responsibility	Key decision
Student web app	Teach-Mitra sessions, notebook upload, concept map, review reminders	Installable web app instead of native apps: one codebase, works on low-end Android; voice and text are equal
Teacher Cockpit	<i>Today's 5</i> , misconception clusters, alerts, calibration queue	Short, evidence-linked cards instead of dense charts
Curriculum service	Concept graph (about 30 concepts for the MVP), required ideas, common misconceptions, probe bank	AI drafts, teacher approves; nothing unapproved reaches students
Mitra engine	Plays the classmate with a seeded misconception	Context contains the misconception script and <i>never</i> the correct answer; an answer-leak check runs on every reply
Peer-execution + verifier	Tests the student's explanation by executing it	Knowledge-ablated: the model sees only the explanation and the problem, labels each step "from explanation" or "assumed," and any "assumed" step marks a gap; the final answer is checked by SymPy, not an LLM
Evidence extractor	Turns transcripts into structured evidence	Schema-validated output; every quote must appear verbatim in the student's text or it is rejected
Notebook reader	Reads handwritten work and finds the first wrong step	Student confirms the reading; low-confidence reads are not counted
Mastery + scheduler	Per-concept status and review dates	Classical statistics and a spaced-repetition model (see B3)
Insight engine	Writes class summaries and <i>Today's 5</i>	Cluster first, summarise second; summaries that fail a "cites real evidence" check are dropped

B3. AI models and techniques

Model tiers (benchmarked on our own Hindi/English evaluation set before choosing)

Task	Tier	Notes
Mitra dialogue, probing	Fast, low-cost model, Hinglish-capable	Small models are often more <i>literal</i> , which suits peer-execution too
Peer-execution	Fast tier, selected by gap-detection rate	If no model passes the day-1 spike, fall back to rubric-only checking
Evidence extraction	Mid/strong tier with reliable structured output	Retry once, then alternate provider, then teacher review queue
Notebook reading	Vision-capable mid tier	If unreadable, ask the student to type the step
Problem generation	Mid tier plus code verifier	The verifier makes it safe; unverified problems are discarded
Offline judging in evals	Strong model from a different provider than the system under test	Teacher labels remain the gold standard

Routing and fallback. A configuration table maps each task to a primary, secondary and tertiary model. The gateway moves down the list on timeout, rate limit or invalid output. If all fail, the student continues with pre-authored practice and evidence capture is queued. Model versions are pinned and changed only through the evaluation gate.

Curriculum retrieval (RAG)

Used for teacher-facing content, hints and rubric references, **not** for Mitra, who must never see curriculum text. Textbook sections (licensing to be confirmed) are split by heading into small chunks tagged with concept IDs, embedded with a multilingual embedding model, and stored in pgvector with hybrid keyword-plus-vector search. Retrieval is tested on a hand-built question-to-passage set in English and Hindi. When confidence is low, the system says "ask your teacher" instead of guessing.

Student knowledge model

- **Concept graph:** prerequisite links, 3-6 required ideas per concept, 3-5 common misconceptions, probes and transfer-problem templates; AI-drafted, teacher-approved.
- **Mastery estimation:** a simple Bayesian update per student, concept and dimension (understanding, retention, transfer). Each piece of evidence is weighted by reliability (lower for uncertain speech reads), independence (lower when hints were used) and recency. A spaced-repetition memory model sets review dates.

- **Prerequisite propagation:** a detected misconception raises the probing priority of weak prerequisites.
- **Baselines:** compared offline against standard Bayesian Knowledge Tracing and an Elo-style update, and reported honestly even if the simple baseline wins. Deep knowledge tracing is deferred until there is enough data.

Speech, handwriting and languages

- **Speech:** Indic-focused and general ASR options (open Indic models, public-sector APIs, Whisper-family, cloud services) are compared on recorded student-like speech. Child speech and Hindi-English mixing are known weak spots, so the app **echoes back "I heard: ..."** for confirmation before anything counts as evidence.
- **Handwriting:** a vision-language model transcribes simple algebra first; geometry diagrams come later.
- **Languages:** English and Hindi (including Hinglish) for the MVP; one or two regional languages in v1. Voice and notebook evidence count only where measured accuracy clears a bar.

Guardrails

Schema validation on every model response with bounded retries; quote grounding for evidence and citation grounding for summaries; SymPy-verified answer keys; an answer-leak check on Mitra; student text always treated as data, never as instructions; moderation and PII scrubbing before external calls.

B4. Data and privacy

Data model (in words). Students, classes and consent records; concepts and their links; sessions and turns; evidence items (with the exact quote and extractor version); per-concept mastery state and next review date; teacher insights, overrides and an audit log. Postgres row-level security limits students to their own data and teachers to their own classes, and consent is a table that gates every student-data endpoint. AI-facing data uses pseudonyms, never real names.

Privacy and security (design intent; legal review required before any pilot).

- Teacher-created accounts, no student email or phone, minimum data stored.
- Parental consent through the school, recorded per student, with a deletion workflow on withdrawal.
- Raw audio deleted after confirmation; notebook images deleted after transcription unless the family opts in.
- No ads, no third-party trackers; AI providers contracted for no training on our data and minimal retention.
- TLS in transit, encryption at rest, least-privilege access, audit log of teacher access to transcripts, data hosted in an India region.
- No real student data in development; synthetic transcripts only.

B5. Key flows

1. A tutoring turn with "Test my explanation." The student's turn passes moderation and PII scrubbing, then Mitra replies using only the misconception script and dialogue (answer-leak check on the way out). When the student taps "Test my explanation," the system generates a fresh problem, verifies its answer key with SymPy, gives the peer model *only the explanation and the problem*, and compares the peer's answer to the key. The failing step is shown to the student ("Mitra got stuck at step 2: what should I do there?").

2. Continuous assessment. When a session ends, a background worker loads the transcript and rubric and runs evidence extraction. Every quote is checked verbatim; invalid or low-confidence results go to a teacher review queue. Valid evidence updates the mastery state and next review date, and a misconception raises prerequisite probing priority.

3. Teacher insight. A nightly or on-demand job applies alert rules (stuck, silent, overconfident, cross-modal mismatch), embeds and clusters misconception quotes, and asks a model to summarise each cluster and suggest an activity. A citation validator drops any summary whose evidence IDs do not exist, and the teacher sees *Today's 5* and the clusters.

B6. Tech stack

Chosen to be proven and operable by one person.

Layer	Choice	Why
Frontend	React, TypeScript, Vite, installable web app, Tailwind	Builder's strongest skill; one codebase for student and teacher views
Backend	Python, FastAPI, Pydantic	Same language as the AI code; typed schemas double as model output contracts
Database	PostgreSQL with pgvector	One store for relational data, vectors and JSON; row-level security
Queue / cache	Redis with a simple Python job library	No heavy infrastructure needed at pilot scale
Storage	S3-compatible bucket with expiring links	Notebook images and temporary audio with auto-delete
Auth	Managed auth with teacher-issued classroom codes	Avoids building auth; supports under-13 students
Hosting	Containers in an India cloud region (single-machine Docker Compose for the pilot)	Data residency, low latency, minimal operations
Observability	Structured logs, error tracking, self-hosted LLM tracing	See every prompt and cost without exposing child data
CI/CD	GitHub Actions: lint, type-check, tests, evaluation gate , secret scan	The evaluation gate lets a solo builder change prompts safely

Cost. Text-only sessions are expected to cost well under a dollar per student per month; speech is the largest cost driver. These are planning estimates to be confirmed against current pricing.

B7. Evaluation plan

Before any child uses the system:

Test	Purpose	Target
Day-1 spike: does knowledge ablation work?	Write about 30 explanations, half deliberately incomplete; check whether the peer fails or marks "assumed" at the right step, and how often it "leaks" outside knowledge	Gap detection at least 80%, leaks at most 10%; if it fails, pivot to rubric-only checking
Evidence accuracy	Compare extraction to two teachers' labels on 200-300 explanations (English, Hindi, Hinglish)	Agreement about 0.7 or better, no worse than teacher-vs-teacher
Mitra safety	Adversarial students ("just tell me the answer," off-topic, distress language)	Answer leaks near zero; all distress cases routed to a human
Problem verification	SymPy check of every generated problem	100% correct keys after verification
Speech and handwriting accuracy	Error rates by language and speaker group	Below threshold means that evidence is not counted
Fairness	Same content in broken vs polished language; accuracy by subgroup	No material score gap
Safety red-team	Prompt injection, PII leakage, inappropriate content	Zero critical failures

Pilot evaluation (later): randomised by class section, pre-registered outcomes, an AI-free post-test, and stop-and-review triggers for any safety incident or worse-than-control results.

B8. Scalability and risks

Risk	Mitigation
50 students start at once	Streaming replies, queue with per-class limits, fast model for dialogue, cached probes
Model outage or deprecation	Gateway fallbacks, pinned versions, evaluation gate before any switch, graceful degradation to pre-authored practice
Core idea fails (peer uses outside knowledge)	Day-1 spike, "assumed" step labels, rubric-only fallback
Weak speech accuracy on children or dialects	Echo-and-confirm, text equal to voice, per-language gating
Slow content authoring	AI drafts, teacher approves, start with one unit
Teacher adoption	Co-design, 3-minute daily view, setup needs only class codes
Rising cost	Model tiering, caching, short contexts, per-student daily caps
Solo-builder risk	Project context stored in the repo for Cline; CI protects quality; small scope

B9. How Cline is used in the build

Cline acts as the rest of the engineering team for a one-person project, under a strict **plan, approve, execute, verify** loop. It handles scaffolding, tests, glue code, refactors and docs. Humans keep ownership of pedagogy, prompt strategy, safety and consent decisions.

Setup

- **.clinerules** files, one per concern, loaded at the start of every task. They encode the project's non-negotiables: all model calls go through the gateway; LLMs never set mastery or control session flow; Mitra never receives answer keys; evidence must carry verbatim quotes; no real student data anywhere; tests first for the mastery math, validators and access rules; ask before adding dependencies.
- **.clineignore** keeps secrets, data folders and audio/image files out of Cline's context.
- **Memory bank** (plain Markdown Cline reads each session): project brief, product context (personas and flows), system patterns (architecture and data model), tech context (stack and commands), active context (current work and next tasks), progress, a decision log and the latest evaluation numbers. This keeps context in the repo, so a solo builder can resume at any time.

Plan Mode vs Act Mode

Plan first (I review and edit before any file changes)	Act directly (after one plan for the pattern)
Database schema, access rules and consent gating	Endpoints that follow an existing pattern
LLM gateway, fallback and budget logic	UI components built from a written spec
Evidence-extraction contract and validators	Unit tests for existing functions
Mastery math and status rules	Synthetic seed data, build scripts
Session state machine, evaluation design	Docs, READMEs, memory-bank updates
Anything touching auth, privacy, prompts, or more than about three files	Refactors protected by passing tests

Loop for each task: Cline reads the memory bank and proposes a plan, I approve or edit it, Cline executes and runs tests after each step, I review the diff, CI and evaluations run, Cline updates the memory bank, then merge.

MCP and tool integrations

- **GitHub:** create issues from the milestone list; open pull requests with an evaluation summary (no direct pushes to main).
- **Read-only Postgres access:** inspect the dev schema and verify migrations and access rules (synthetic data only).
- **Docs lookup:** fetch current documentation for FastAPI, pgvector, Pydantic and the scheduling library, so Cline does not guess APIs from memory.
- **Browser automation:** open the running app, click through Teach-Mitra and the cockpit, take screenshots and write end-to-end tests.
- **Custom content server (small, written with Cline's help):** read-only access to the approved concept graph and rubrics so generated tests and seed data match approved content.

Safeguards against blindly trusting generated code

1. **Checkpoints** before each task, to compare and roll back.
2. **Line-by-line review** of auth, access rules, consent, prompts and the mastery module (auto-approve off for these).
3. **Command allowlist** for read-only and test commands; no deploy or destructive commands.

4. **Tests first** for mastery math, quote and citation validators, and access rules.
5. **CI is the judge**, not the agent's own summary: lint, types, tests, evaluation gate, secret scan, dependency audit.
6. **Second-opinion review** by a fresh Plan-mode session (ideally a different model) before merge.
7. **Dependency discipline**: verify every new package exists and is maintained, then pin versions.
8. **Human-owned decisions**: pedagogy, prompt strategy, safety and consent text are decided by me; Cline drafts, but does not decide.

Build roadmap

Stage	Cline does	Human owns	Done when
Day 1-2: Spike and setup	Repo scaffold, rules, memory bank, script that runs peer-execution on the 30 test explanations and reports gap-detection and leak rates	Writing the explanations; judging results; go/no-go on the core idea	Spike results recorded
Foundations	API skeleton, database and access rules with tests, classroom-code login, CI, UI shell	Threat model; review of access and consent code	A student cannot read another student's data (tested)
Curriculum	Concept-graph format, validators, approval UI, retrieval pipeline	Concept graph design, misconception library, rubrics (with a teacher)	One unit of about 30 concepts approved
Mitra and peer-execution	Gateway, state machine, Mitra service, peer-execution, SymPy sandbox, streaming	Prompt strategy, safety review of dialogues	Teach-Mitra works end to end
Evidence and mastery	Extractor with quote validator, mastery updates, review scheduling, workers, student UI	Rubric calibration, status thresholds, labelling explanations	"Mitra forgot" loop works
Teacher cockpit	Insight pipeline, <i>Today's 5</i> , cockpit UI, end-to-end tests	Teacher interviews, card wording	Teacher can act on a card in under 3 minutes
Voice, notebook, Hindi	Speech integration, notebook pipeline, translation scaffolding	Recording speech samples; per-language go/no-go	Gated by measured accuracy
Hardening and pilot readiness	Red-team tests, load tests, consent and deletion flows, demo data	Safety, privacy and legal review; pilot protocol	Pilot can start

Minimum viable build (hackathon scope)

One unit (Grade 8 linear equations, about 30 concepts) in English and Hindi **text**: Teach-Mitra with a seeded misconception, "Test my explanation," quote-grounded evidence, mastery statuses with one retention return ("Mitra forgot"), and a teacher view with a misconception cluster and *Today's 5* on **synthetic** class data. Voice, notebook photos and parent messages are stretch goals.

Non-goals: an answer-giving chatbot, replacing teachers or board exams, student rankings or leaderboards, AI-text detectors, streaks and badges, ads or behavioural tracking, and training a foundation model.